

BL PCF Report

Technical Guide

Predator Compensation Fund — Livestock Predation Incident Analysis

Version 2.0

Generated August 13, 2026

Workflow id: **bl_pcf_report**

1. Overview

The **bl_pcf_report** workflow ingests livestock predation events (event type **hwc_lvstprd**) from EarthRanger for the Amboseli ecosystem and produces a comprehensive Predator Compensation Fund (PCF) incident analysis report and dashboard. The workflow covers three target ranches — **Eselengei**, **Mbirikani**, and **Kimana** — and restricts analysis to **valid claims** only.

Note: Version 2.1 removes the `gee_client` connection field, which was dropped from `spec.yaml` — this workflow never used it (it never performed NDVI or other Earth-Engine analysis). Version 2.0 rewrote this guide against the (then-current) `spec.yaml`, correcting the dependency list (migrated to `ecoscope-platform`), the actual chart/task names (several task and function names in the original version did not match any task in `spec.yaml`), the previous-period mechanism (a flexible Custom/Preset/Calendar offset, not a simple integer), the Word report's fixed "Ecoscope" author field and `overall_report.docx` filename, and the dashboard, which wires up 24 widgets rather than an empty list.

For each run the workflow delivers:

- 14 distinct chart-drawing steps, producing 19 chart files (some fan out per ranch) — pie, stacked bar, time-of-day bar, multi-line and multi-bar time series, and a faceted historic comparison chart per ranch
- 3 maps — overall predation density grid, boma-attack density grid, and a livestock-species scatter map
- 13 summary/breakdown tables (GeoParquet + HTML) — an overall summary, one per-ranch summary per ranch, and breakdowns by claim type, predator, attack location, and boma type
- 2 raw event data files (CSV) — cleaned current- and previous-period events
- A Word document report (`overall_report.docx`) — every chart, map, and table assembled into the Big Life PCF report template
- A 24-widget interactive dashboard

Output summary

Output type	Count	Description
Pie charts	5	Killed & compensation by predator, compensation by ranch, attack location, boma type targeted
Stacked bar charts	3	Killed & claim count by claim type × ranch; 100%-stacked killed by predator × ranch
Time-of-day bar chart	1	Predation incidents by 4-hour time bin
Multi-line time series	3	Killed over time by ranch, by attack location, and claim count over time by type
Multi-bar time series	1	Animals killed over time per predator (faceted, 2-column grid)
Historic comparison chart	1 task → 3 files	Current vs. historic mean with 95% CI, faceted by predator, one file per ranch
Density grid maps	2	All predation incidents; boma attacks only
Scatter map	1	Livestock species scatter map
Summary/breakdown tables (GeoParquet + HTML)	13	Overall, 3× per-ranch, claim type, predator, location, predator×location, boma

Output type	Count	Description
Raw event data (CSV)	2	current_events.csv, previous_events.csv
Word document	1	overall_report.docx
Dashboard widgets	24	Every chart, map, and table above; ranch-level charts/tables merged into 2 switchable widgets

2. Dependencies

2.1 Python packages

The workflow declares eight versioned packages, resolved from the Ecoscope prefix.dev channels:

Package	Version	Channel
ecoscope-platform	2.18.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.0.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0,<2.0.0	conda-forge

The chart-drawing tasks (pie, bar, multi-line, multi-bar, faceted historic time series) come from ecoscope-workflows-ext-wwf-virunga. fill_missing_values and the integer-conversion helper come from ecoscope-workflows-ext-mnc. The Dropbox fetch, previous-period calculation, envelope/view-state, and map-layer combination tasks come from ecoscope-workflows-ext-ste. The Word-report generation task comes from ecoscope-workflows-ext-big-life.

2.2 External data — static Amboseli layers

Three GeoPackage files are downloaded from Dropbox at run time and cached locally (overwrite_existing: false, 3 retries):

File	Purpose
amboseli_ranch_conservancies_layers.gpkg	Land-use styling layer overlaid on all maps
amboseli_group_ranch_boundaries_x_electric_fence.gpkg	Ranch boundaries and electric fence overlay
amboseli_group_ranch_boundaries.gpkg	Conservancy boundary polygons (reprojected, not otherwise used downstream)

2.3 Base map tiles

Two ESRI tile layers are composited for every map:

Layer	Opacity	Max zoom
ESRI World Hillshade	1.0	20
ESRI World Street Map	0.15	20

2.4 EarthRanger connection

A single EarthRanger connection (set_er_connection) is required and reused for both the current- and previous-period event fetches.

2.5 Grouper

The workflow groups data by the **Ranch** column (fixed via set_groupers, not user-configurable). This grouper drives every per-ranch table split and the ranch-level historic chart fan-out.

2.6 Time frequency

A user-selectable time_frequency parameter (select_time_frequency — Annual / Monthly / Weekly / Daily) controls the temporal aggregation unit used by all multi-line and multi-bar time-series charts, and by the historic comparison chart.

3. Data Ingestion and Processing

3.1 Dual pipeline — current and previous period

Events of type `hwc_lvstprd` are fetched twice: once for the selected time range (`get_current_events`, `raise_on_empty`: `true`) and once for the comparison period (`get_previous_events`, `raise_on_empty`: `false`, so an empty previous period doesn't fail the run). Both event sets pass through an identical transformation chain before diverging into the overall/per-ranch summaries and the historic comparison chart.

3.2 Previous period calculation

The comparison window is computed by `ecoscope_workflows_ext_ste.tasks.filter.flexible_previous_period` (task id `set_previous_period`), which always ends the comparison period on the current time range's start date, then works backward. The user chooses one of three offset modes in the form: **Custom** (Years/Months/Weeks/Days, default 1 month), **Preset** (e.g. 1/3/6 months or 1 year back), or **Calendar** (an exact start date).

3.3 Field normalisation

Each pipeline (current and previous) applies the same sequence:

Step	Task	Purpose
1	<code>process_events_details</code>	Maps EarthRanger detail keys to their display titles (<code>map_to_titles</code> : <code>true</code> , <code>ordered</code> : <code>true</code>)
2	<code>normalize_json_column</code>	Flattens the <code>event_details</code> JSON column into individual <code>event_details__<title></code> columns
3	<code>drop_column_prefix</code> (×2)	Strips the <code>event_details__</code> prefix, then the <code>hwc_lvstprd__</code> prefix, <code>duplicate_strategy</code> : <code>keep_original</code>
4	<code>map_columns</code>	Drops ~30 unused detail columns (header info, verification metadata, boma coordinates, etc.); retains the rest as-is

3.4 Filtering

Two filters are applied in sequence, via `filter_row_values`:

- **Ranch filter** — retains only rows where Ranch is one of Eselengei, Mbirikani, or Kimana
- **Validity filter** — retains only rows where "Validity of claim" == "Valid"

3.5 Livestock-kill totals

The adult and young kill counts are each combined from two possible source columns via `create_combined_column` (primary column, with a fallback if the primary is null, `fill_remaining`: 0):

Output column	Primary column	Fallback column
<code>total_adult_livestock_killed</code>	Adults killed	<code>hwc_lvstpd_number_adults_killed</code>
<code>total_young_livestock_killed</code>	Young (<1yr) killed	<code>hwc_lvstprd_<1yr</code>

`convert_column_values_to_numeric` then casts `Compensation` value to `owner`, `total_adult_livestock_killed`, and `total_young_livestock_killed` to numeric, and `fill_missing_values` fills any remaining nulls in those three columns with 0.

A second `fill_missing_values` pass fills nulls in `Animal responsible`, `Boma type`, `Livestock species`, `Type of claim`, and `Where were the livestock when the attack happened` with the string "Unknown".

Finally, `apply_arithmetic_operation_over_rows` (operation: add) computes the column every downstream chart and table is built from:

```
total_animals_killed = total_adult_livestock_killed + total_young_livestock_killed
```

The cleaned current- and previous-period tables are persisted as `current_events.csv` and `previous_events.csv`.

4. Summary Tables

4.1 Overall predation summary

Three crosstabs — incidents (count), animals killed (sum), and compensation value (sum), each indexed by Animal responsible with Livestock species as columns and a Total margin — are computed from the current-period events via `crosstab_summary` and combined into one table by `summarize_predation_table` (`overall_summary_table`), then persisted as GeoParquet (`overall_predation_summary`).

4.2 Per-ranch summaries

`split_groups` partitions the current-period events by Ranch (`split_by_ranch`). For each ranch, the same three crosstabs are computed via `mapvalues` and combined with `summarize_predation_table`. The ranch name for each partition is extracted with `column_first_unique_value` and zipped onto its summary table (`groupbykey`), so each ranch's table is persisted under a filename equal to the ranch name itself (e.g. a file literally named `Eselengei`).

4.3 Breakdown tables

All computed from the current-period events via `crosstab_summary`, then formatted with `convert_columns_to_int` and `format_numbers_with_commas`:

Output id	Rows × Columns	Value
<code>livestock_killed_by_claim_type</code>	Type of claim × Ranch	<code>sum(total_animals_killed)</code>
<code>livestock_killed_by_predator_species</code>	Animal responsible × Ranch	<code>sum(total_animals_killed)</code>
<code>livestock_attacks_by_location</code>	attack location (single dimension)	count + percentage
<code>predation_incidents_by_predator_and_location</code>	Animal responsible × attack location	count of id, merged with per-predator incident totals and percentages
<code>predation_incidents_by_boma_type</code>	Boma type (single dimension)	count + percentage

5. Color Mapping

Six colour maps are assigned via `map_column_value`, each adding a dedicated hex-colour column to the current-period events `GeoDataFrame`. `add_rgba_columns_from_hex` then converts all six into `<column>_rgba` variants — it is the `_rgba` columns, not the raw hex columns, that are wired into the chart and `map_color_column` parameters.

5.1 Animal responsible

Animal	Hex color	Animal	Hex color
Caracal	#fd7f6f	Leopard	#ffee65
Cheetah	#7eb0d5	Lion	#beb9db
Elephant	#b2e061	Unknown	#cfcfc4
Hyena	#bd7ebe	Wild dog	#4cc9b0
Jackal	#ffb55a	—	—

5.2 Other color maps

Color map	Key	Hex color
Boma type	Permanent	#fd7f6f
Boma type	Temporary	#7eb0d5
Livestock location	Inside Boma	#fd7f6f
Livestock location	More than 200m from Boma	#7eb0d5
Livestock location	Within 200m of Boma	#b2e061
Claim type	Bad Boma	#beb9db
Claim type	No Penalty	#b2e061
Claim type	Lost in the Bush	#7eb0d5
Ranch	Eselengei	#fd7f6f
Ranch	Kimana	#7eb0d5
Ranch	Mbirikani	#b2e061
Livestock species	Shoat	#0000ff
Livestock species	Cow	#8b0000
Livestock species	Donkey	#ffff00

Any category value not present in a mapping falls back to `#f0f8ff` (`keep_unmapped: false` drops rows whose value has no explicit mapping and isn't the default).

6. Charts

6.1 Pie charts

All five pie charts use `ecoscope_workflows_ext_wwf_virunga.tasks.plot._plot.draw_pie_chart` with `textinfo: "percent+label+value"`, `font_size: 12`, `showlegend: true`.

Output file	Value column	Label / colour column
livestock_killed_by_predator_pie	total_animals_killed	Animal responsible / animal_responsible_colors_rgba
compensation_value_by_predator_pie	Compensation value to owner	Animal responsible / animal_responsible_colors_rgba
compensation_value_by_ranch_pie	Compensation value to owner	Ranch / ranch_colors_rgba
livestock_attack_location_pie	Where were the livestock... (categorical)	livestock_attack_colors_rgba
boma_type_targeted_pie	Boma type (categorical)	boma_type_colors_rgba

6.2 Stacked bar charts

Three charts use `ecoscope_workflows_ext_wwf_virunga.tasks.plot.draw_bar_chart` (`mode: stacked` or `percent_stacked`), category axis Ranch, `plot_bgcolor: #f5f5f5`, `bargap/bargroupgap: 0.05`.

Output file	Y axis / agg	Stack column	Stack order
livestock_killed_by_claim_type_bar	total_animals_killed / sum	Type of claim	Lost in the Bush → No Penalty → Bad Boma
claim_count_by_type_bar	id / count	Type of claim	Lost in the Bush → No Penalty → Bad Boma
livestock_killed_by_predator_pct_bar	total_animals_killed / sum (percent_stacked)	Animal responsible	Hyena → Jackal → Lion

6.3 Time-of-day bar chart

Built with `draw_bar_chart` (`mode: grouped`, category: "Time of attack", `agg_func count`, `showlegend: false`, bar colour `#6495ed`).

Time bin	Hours covered
Morning	06:00 – 11:59
Afternoon	12:00 – 16:59
Evening	17:00 – 19:59
Night	20:00 – 05:59

Output: **predation_incidents_by_time_of_day_bar**

6.4 Multi-line time-series charts

Three charts use `draw_grouped_line_time_series_chart`, all driven by the user-selected `time_frequency`.

Output file	Group column	Y / agg	Fill	Group order
livestock_killed_over_time_by_ranch_chart	Ranch	total_animals_killed / sum	No	Eselengei → Mbirikani → Kimana
livestock_killed_over_time_by_attack_location_chart	attack location	total_animals_killed / sum	Yes	Inside Boma → Within 200m → More than 200m
claim_count_over_time_by_type_chart	Type of claim	id / count	Yes	Lost in the Bush → No Penalty → Bad Boma

6.5 Multi-bar time-series chart (faceted per predator)

Built with `draw_faceted_bar_time_series_chart`: `total_animals_killed` summed per Animal responsible, in a 2-column subplot grid (`ncols`: 2, `row_height`: 350, `shared_yaxes`: false), bar colour #6495ed. Screenshot: 1280 × 2000 px.

Output: **livestock_killed_over_time_by_predator_multibar_chart**

6.6 Historic comparison chart (per ranch)

`generate_grouped_historical_stats_table` is mapped once per ranch over the current/previous partitions (`group_ranch_level_groupers` zips `split_by_ranch` with `split_previous_by_ranch`), producing a stats table per ranch with a 95% confidence interval (`band_method`: `ci_mean`, `band_level`: 0.95) around the historic mean of `total_animals_killed`, grouped by Animal responsible. `draw_faceted_historic_timeseries` then renders one faceted chart per ranch (`facet` = `predator`, `ncols`: 2, `row_height`: 300):

Visual element	Column	Colour
Current value line	current_value	rgb(0, 0, 139) — dark blue
Historic mean line	historic_mean	#ff8c00 — dark orange
Historic 95% CI band	historic_min / historic_max	rgb(143, 188, 139, 0.5) — translucent sage green

Outputs: **ranch_level_historic_time_series_chart_<ranch>** — one file per ranch, filename suffixed with the ranch name (via `groupbykey` zipping the ranch name onto the rendered chart). Screenshot: 1280 × 2000 px.

7. Maps

All three maps share the same base-tile stack (Hillshade + Street Map) and the same three Amboseli overlay layers (land use, ranch boundaries, electric fence) as static layers, framed by a shared view state (envelope_gdf, expansion factor 1.05; compute_view_state_from_gdf, pitch 0, bearing 0, max zoom 15). Map screenshots use device_scale_factor: 2.0 and wait_for_timeout: 40000 ms to allow the base tiles to fully render.

7.1 Livestock predation event map

create_scatterplot_layer plots every current-period event as a point, coloured by livestock_species_colors_rgba (get_radius: 4, opacity 0.75). Legend sorted ascending by livestock species.

Livestock species	Colour
Shoat	#0000ff (blue)
Cow	#8b0000 (dark red)
Donkey	#ffff00 (yellow)

Output: **livestock_predation_event_map**

7.2 Predation incident density map

All current-period events are gridded into a 2000 m mesh (create_meshgrid, EPSG:3857) and a per-cell point count (calculate_feature_density). Zero-density cells are dropped, and the remaining density values are classified into 5 equal-interval bins (apply_classification) and coloured yellow-to-dark-red:

Bin rank	Hex colour
1 (lowest)	#FFF7BC
2	#FD8D3C
3	#F03B20
4	#BD0026
5 (highest)	#99000D

Grid cells are rendered as filled GeoJSON polygons (opacity 0.55, black outline, 0.35 px line width). Output: **predation_incident_density_map**

7.3 Boma predation density map

Identical pipeline to §7.2, but events are first filtered to "Where were the livestock when the attack happened" == "Inside Boma" before gridding. Same 2000 m mesh, equal-interval k=5 classification, and colour ramp.

Output: **boma_predation_density_map**

8. Word Report & Dashboard

8.1 Template

The Word template `pcf_report_template.docx` is downloaded from Dropbox at run time (`overwrite_existing: false`, 3 retries) via `fetch_and_persist_file`.

8.2 Report generation

`generate_pcf_report` populates the template with every chart, map, and table:

Field	Value
<code>template_path</code>	downloaded <code>pcf_report_template.docx</code>
<code>output_dir</code>	<code>\$ECOSCOPE_WORKFLOWS_RESULTS</code>
<code>filename</code>	<code>overall_report.docx</code>
<code>time_period</code>	the workflow's current time range
<code>generated_by</code>	"Ecoscope" (fixed string — not resolved from the EarthRanger user)
<code>validate_images</code>	true — verifies every expected image exists before populating

8.3 Dashboard

`gather_dashboard` registers **24 widgets** — every chart, map, and table produced by the run. Two of them are merged multi-view widgets built with `merge_widget_views`:

- **Historic Predation Trend by Ranch** — the 3 per-ranch historic comparison charts, merged into a single widget the viewer can switch between
- **Predation Summary by Ranch** — the 3 per-ranch summary tables, merged the same way

The remaining 22 widgets are single-view plot, map, and table widgets — one per chart/map/breakdown table described in §6 and §7, plus the overall summary table and the four breakdown tables from §4.3.

9. Output Files

All outputs are written to **\$ECOSCOPE_WORKFLOWS_RESULTS**:

File	Description
current_events.csv / previous_events.csv	Cleaned, filtered current- and previous-period event tables
overall_predation_summary (GeoParquet + .html)	Incidents, killed, compensation by predator, overall
<ranch> (GeoParquet + .html, one per ranch, named after the ranch)	The same summary, per ranch
livestock_killed_by_claim_type (GeoParquet + .html)	Animals killed by claim type × ranch
livestock_killed_by_predator_species (GeoParquet + .html)	Animals killed by predator × ranch
livestock_attacks_by_location (GeoParquet + .html)	Incident count/pct by attack location
predation_incidents_by_predator_and_location (GeoParquet + .html)	Incident count/pct by predator × attack location
predation_incidents_by_boma_type (GeoParquet + .html)	Incident count/pct by boma type
livestock_killed_by_predator_pie compensation_value_by_predator_pie compensation_value_by_ranch_pie livestock_attack_location_pie boma_type_targeted_pie (.html + .png)	/ / / / / 5 pie charts — see §6.1
livestock_killed_by_claim_type_bar claim_count_by_type_bar livestock_killed_by_predator_pct_bar (.html + .png)	/ / / 3 stacked bar charts — see §6.2
predation_incidents_by_time_of_day_bar (.html + .png)	Time-of-day bar chart — see §6.3
livestock_killed_over_time_by_ranch_chart / livestock_killed_over_time_by_attack_location_chart / claim_count_over_time_by_type_chart (.html + .png)	3 multi-line time series — see §6.4
livestock_killed_over_time_by_predator_multibar_chart (.html + .png)	Faceted multi-bar time series — see §6.5
ranch_level_historic_time_series_chart_<ranch> (.html + .png, one per ranch)	Historic comparison chart — see §6.6
livestock_predation_event_map (.html + .png)	Scatter map — see §7.1

File	Description
predation_incident_density_map (.html + .png)	Density grid — see §7.2
boma_predation_density_map (.html + .png)	Density grid — see §7.3
overall_report.docx	Final populated Word PCF report
amboseli_ranch_conservancies_layers.gpkg / amboseli_group_ranch_boundaries_x_electric_fence.gpkg / amboseli_group_ranch_boundaries.gpkg / pcf_report_template.docx	Cached inputs downloaded from Dropbox

10. Workflow Execution Logic

10.1 Skip conditions

Every task carries the two default skip conditions from task-instance-defaults: **any_is_empty_df** (skip if any input DataFrame is empty) and **any_dependency_skipped** (skip if an upstream task was skipped). This propagates gracefully through the pipeline — if no valid events exist for a ranch or for the previous period, all downstream tasks in that branch are skipped rather than raising an error.

The current-period event fetch sets `raise_on_empty: true` (a zero-event current period is treated as a configuration/data problem worth surfacing), while the previous-period fetch sets `raise_on_empty: false` (a missing comparison period is expected and handled gracefully).

10.2 mapvalues / map fan-out

mapvalues and map directives fan several steps out per ranch:

- **split_by_ranch / split_previous_by_ranch** — partition the current/previous events by Ranch
- **predation_ranch_incidents / _killed / _compensation** — the three crosstabs, computed once per ranch partition
- **get_ranch_name** — extracts the ranch name string from each partition, later zipped onto that ranch's table/chart output to control its filename suffix
- **hist_curr_prev_table / draw_ranch_level_historic_chart** — the historic stats table and chart, computed once per ranch from the zipped current+previous partitions
- **widget_ranch_historic_chart / widget_ranch_summary_table** — per-ranch dashboard widgets, later merged into 2 switchable widgets via `merge_widget_views`

10.3 Screenshot timing

Task(s)	wait_for_timeout	Notes
convert_chart_html_png (12 charts, batched)	1 ms	1280×720; static Plotly HTML renders essentially instantly
convert_pred_killed_multibar_png	10 ms	1280×2000
convert_ranch_historic_chart_png (×3, one per ranch)	10 ms	1280×2000
convert_livestock_map_png / convert_density_map_png / convert_density_boma_map_png	40 000 ms	Maps require full basemap tile rendering before capture

Note: The batched `convert_chart_html_png` task's `html_path` list contains `persist_killed_pred_stacked_bar` twice in `spec.yaml` — a harmless duplicate entry, not a second distinct chart.

10.4 Geometry cleaning

No outlier or null-geometry filtering is applied before the livestock scatter map in the current pipeline — every valid-claim event with a point geometry is plotted directly via `create_scatterplot_layer`.

11. Software Versions

Package	Version pinned	Channel
ecoscope-platform	2.18.0	ecoscope-workflows
ecoscope-workflows-ext-custom	0.1.0rc14.*	ecoscope-workflows-custom
ecoscope-workflows-ext-ste	0.0.0rc1.*	ecoscope-workflows-custom
ecoscope-workflows-ext-mnc	1.0.0.*	ecoscope-workflows-custom
ecoscope-workflows-ext-wwf-virunga	0.0.0rc9.*	ecoscope-workflows-custom
ecoscope-workflows-ext-big-life	1.0.0.*	ecoscope-workflows-custom
pydeck	0.9.2	conda-forge
opentelemetry-sdk	>=1.20.0,<2.0.0	conda-forge

Note: All Ecoscope packages are resolved from the prefix.dev ecoscope-workflows / ecoscope-workflows-custom channels. Wildcard patch/prerelease pins (.*) allow bug-fix releases to be picked up automatically while keeping the major/minor version — and, for the rc-pinned packages, the prerelease line — locked. The runtime environment is managed by **pixi**.